

오늘의 작업기록 — 2026년 6월 24일

진행 추적 대시보드 구축 + GitHub Pages 공개 배포

(중분 3 대신, 학습을 추적할 시스템을 만들고 공개한 날)

한눈에

항목	내용
오늘 한 일	코딩(중분 3) 대신 메타 도구 구축 — 9개월 진행을 추적·공개하는 대시보드
결과물	Backend Journey 대시보드 + GitHub 공개 배포 + 자동 갱신 파이프라인
공개 URL	taewoo-won.github.io/backend-journey/ (영구)
코드 저장소	github.com/Taewoo-Won/backend-journey (Public)
처음 한 일	프로젝트 전체를 git으로 GitHub에 푸시 + GitHub Pages 배포 (둘 다 처음)
핵심 개념	단일 진실 공급원 · git 4단계 · GitHub Pages · 자동화(cron) · 멍등성
왜 의미 있나	배포·자동화·git은 그 자체로 백엔드 실무 역량. 그리고 이 대시보드 = 공개 포트폴리오 자산

오늘은 자바 한 줄 안 짰다. 대신 **앞으로 9개월을 추적할 시스템**을 만들고, 그걸 인터넷에 공개했다. 배포와 자동화는 백엔드 개발자가 결국 해야 하는 일이고, 오늘 그 첫 삽을 떴다.

0. 오늘은 왜 코딩을 안 했나

중분 3(커스텀 예외)은 다음으로 미뤘다. 대신 더 시급한 걸 했다 — **진행 상황을 한눈에 보고, 매일의 작업기록을 한곳에 모으고, 그걸 공개해서 증거로 남기는 도구**.

이건 결과지가 아니다. 포트폴리오 전략의 두 원칙을 그대로 실행한 거다: - **증거를 남겨라** — 매일 쌓이는 작업기록 + 가시화된 궤적. - **공개로 발견되게 하라** — GitHub Pages 공개 URL.

그리고 오늘 만지는 git·배포·자동화는 전부 실무 백엔드 스킬이다.

PART 1 — 무엇을 만들었나 (대시보드)

1-1. 핵심 설계: 데이터와 화면을 분리한다

가장 중요한 결정. 진행 데이터와 보여주는 화면을 **따로** 둔다.

roadmap.json → build.py → index.html
(데이터) (반영기) (화면)

- roadmap.json** — 단일 진실 공급원(single source of truth). 7개 레이어, 34개 마일스톤, 각각의 상태(todo/in_progress/done)와 무게(weight), 작업기록 목록이 전부 여기 있다.
- build.py** — roadmap.json을 읽어서 index.html 안에 끼워 넣는다.
- index.html** — 게이지·타임라인·카드를 그리는 화면. 데이터는 안 갖고 있고, build가 넣어준 걸 그릴 뿐.

왜 분리하나? 진행 상황이 바뀌면 **roadmap.json 한 곳만** 고치면 된다. 화면(HTML)은 손 안 댄다. 데이터와 표현을 섞으면, 숫자 하나 바꾸려고 화면 코드를 헤집어야 한다. 이 분리가 나중에 백엔드에서 배울 "관심사 분리"의 가장 작은 형태다.

1-2. 진행률은 개수가 아니라 무게로 센다

마일스톤마다 **weight** (무게)가 있다. WAL·동시성 같은 큰 산은 6, 자격증은 2. 전체 진행률은:

진행률 = (끝낸 무게의 합) ÷ (전체 무게의 합)
· 완료=무게 100%, 진행중=무게 50%, 할일=0%

오늘 기준 7%. 단순히 "34개 중 2개=6%"가 아니라, 무게를 반영한 정직한 숫자다. WAL 같은 천장을 넘으면 %가 크게 뛴다.

1-3. 작업기록 아카이브 (md + pdf)

대시보드 맨 아래에 날짜별 작업기록 카드. 각 카드에 **MD·PDF 버튼**이 있어서, 그날 기록(19~23일 + 오늘)을 바로 열 수 있다.

1-4. PDF 파이프라인 (한글 깨짐 해결)

작업기록 md를 전문적인 PDF로 바꾸는 변환기를 만들었다:

- 한글은 **Noto Sans CJK KR** 폰트로 (안 그러면 네모(□)로 깨진다).
- 코드 블록·표·헤더에 CSS로 전문 스타일 적용.
- 이모지(✓×)는 PDF에서 깨지므로 ✓× 같은 기호로 자동 치환.

PART 2 — GitHub에 공개 배포 (오늘 친 명령어 전부)

오늘 처음으로 프로젝트 전체를 인터넷에 올렸다. 친 명령어와 각각의 뜻:

2-1. 파일을 VPS로 (SFTP)

대시보드 묶음(zip)을 Termius **SFTP**로 VPS에 업로드 → **unzip** 으로 풀기. - **SFTP** = SSH 위에서 파일을 주고받는 방식. Termius 안에 내장.

```
unzip -o backend-journey.zip # -o = 같은 파일 있으면 덮어쓰기
```

2-2. git 4단계 (작업물 → GitHub)

```
git init # ① 이 폴더를 git 저장소로
git add . # ② 모든 변경을 '스테이징'에 올림
git commit -m "backend journey ..." # ③ 스냅샷으로 '확정' (로컬에 기록)
git branch -M main # 기본 브랜치 이름을 main으로
```

git의 흐름은 작업폴더 → 스테이징 → 로컬 저장소 → 원격(GitHub) 4칸이다. - **add** = 작업폴더 → 스테이징 (올릴 것 고르기) - **commit** = 스테이징 → 로컬 (확정, 메시지로 "뭘 했는지" 기록) - **push** = 로컬 → GitHub (인터넷에 반영) ← 아래 2-3에서

2-3. 공개 저장소 생성 + 푸시 (한 방에)

```
gh repo create backend-journey --public --source=. --push
```

- **gh** = GitHub 공식 CLI. 이미 로그인돼 있음.
- **--public** = 누구나 볼 수 있는 공개 저장소
- **--source=.** = 지금 이 폴더를
- **--push** = 만들자마자 GitHub로 push까지

→ 저장소 생성 + 원격 연결 + 첫 push가 한 줄에 끝났다.

2-4. GitHub Pages 켜기 (저장소 → 웹사이트)

```
echo '{"source":{"branch":"main","path":"/"}}' \
| gh api -X POST repos/Taewoo-Won/backend-journey/pages --input -
```

- **GitHub Pages** = 저장소의 정적 파일(HTML 등)을 **웹사이트**로 띄워주는 GitHub 기능. 무료.
- main 브랜치의 루트(/)를 사이트 소스로 지정.

→ 1~2분 뒤 taewoo-won.github.io/backend-journey/ 에서 대시보드가 뜬다.

2-5. ★ 헛갈린 핵심: 주소가 두 개다

주소	무엇
github.com/Taewoo-Won/backend-journey	코드 참고 — 파일 목록(index.html, roadmap.json…)
taewoo-won.github.io/backend-journey/	실제 화면 — 게이지·타임라인이 뜨는 대시보드

둘 다 GitHub이 띄운다. **.github.io** 는 GitHub Pages 전용 도메인이고, 주소는 **자동으로 소문자가 된다**(아이디가 **Taewoo-Won** 이어도 **taewoo-won**). 평소 보는 건 **.github.io** 쪽.

PART 3 — 공개 전 보안·전문성 정리

공개 저장소 = 전 세계 누구나 본다. 그래서 올리기 전에 두 가지를 정리했다.

3-1. 민감 정보 빼기 (VPS IP)

작업기록 안에 내 **VPS 공인 IP**가 적혀 있었다. 공개되면 누구나 본다. → 전 파일에서 제거하고, PDF까지 다시 생성해서 텍스트로도 안 남게 했다.

교훈: 공개 저장소에 올리기 전엔 항상 **민감 정보(IP·키·비밀번호·내부 주소)**가 없는지 훑는다. 키 기반 SSH라 위험은 낮았지만, 습관이 중요하다.

3-2. 표현을 전문 톤으로

처음엔 대화하듯 쓴 문구·과한 표현이 섞여 있었다. 공개 문서는 **남이 보는 글**이다. → 부제·레이어 이름·README를 깔끔한 기술 문서 톤으로 다시 썼다. (대화체 → 객관 서술)

교훈: 공개 레포의 README·문서는 곧 명함이다. 채용담당자가 본다는 전제로 쓴다.

PART 4 — 자동 갱신 파이프라인 (cron + update.py)

4-1. 문제

매일 작업기록을 올릴 때마다 직접: 파일 넣기 → roadmap.json 고치기 → build → commit → push. **귀찮다**. 단계가 많으면 안 하게 된다.

4-2. 해결: 자동 감시

update.py 스크립트 + **cron**(주기 실행기)을 조합했다.

```
cron 이 1분마다 update.py 실행
  → logs/ 에서 새 작업기록(.md) 자동 감지
  → roadmap.json 에 항목 추가
  → build.py 로 대시보드 갱신
  → 변경 있으면 git commit + push
```

이제 **logs/에 md+pdf 두 파일만 올리면**, 1분 안에 공개 URL이 자동 갱신된다. 명령어도, 날짜 폴더도 안 만든다.

cron 설치(한 번만):

```
( crontab -l 2>/dev/null | grep -v 'backend-journey'; \
  echo '* * * * * cd /root/backend-journey && /usr/bin/python3 update.py >> /root/backend-journey/.cron.log 2>&1' ) | crontab -
```

- *** * * * *** = 매분 실행.

4-3. ★ 멍등성 — 로드맵에서 배울 그 개념을 오늘 썼다

자동 감시가 1분마다 도는데, **새 파일이 없으면 아무것도 안 해야** 한다. 안 그러면 1분마다 의미 없는 커밋이 쌓인다.

그래서 update.py는: **실제 변경이 있을 때만 커밋·푸시한다**. 같은 상태로 100번 실행해도 결과가 똑같다(커밋 0개). 이게 **멍등성(idempotency)** 이다.

이 단어, 로드맵 Layer 1·2에 그대로 나온다 — "멍등성 키: 재시도하는 요청이 두 번 실행 못 하게." 결제에서 같은 이체 요청이 두 번 와도 한 번만 처리되게 하는 그 개념. **오늘 내 자동화 도구에서 먼저 손맛을 봤다**. 나중에 결제 원장에서 만날 때 낯설지 않을 거다.

4-4. 카드 메타블록 (md 안에 숨긴 데이터)

작업기록 카드의 제목·요약을 자동으로 채우려고, md 파일 맨 위에 **숨겨진 메타블록**을 둔다:

```
<!-- -->
```

는 HTML 주석이라 PDF·화면엔 **안 보인다**. update.py만 이걸 읽어서 카드에 넣는다. (이 작업기록 파일 맨 위에도 들어있다.)

5. 산출물

- 공개 대시보드: **taewoo-won.github.io/backend-journey/**
- 코드 저장소: **github.com/Taewoo-Won/backend-journey** (Public)
- 구성:
- **index.html** — 대시보드 화면
- **roadmap.json** — 단일 진실 공급원 (진행 데이터 + 작업기록 목록)
- **build.py** — roadmap.json → index.html 반영
- **make-pdf.py** — 작업기록 md → pdf
- **update.py** — 자동 감지·갱신·커밋·푸시
- **logs/** — 일자별 작업기록 (md + pdf)
- **자동화:** cron 1분 주기 → logs/ 감시 → 자동 배포

6. 오늘 배운 것 (개념 사전)

- 단일 진실 공급원 (**single source of truth**): 데이터를 한 곳(roadmap.json)에만 둔다. 화면은 그걸 그릴 뿐, 고칠 땐 한 곳만.
- 관심사 분리: 데이터(json) / 반영(build.py) / 화면(html)을 나눈다. 백엔드 설계의 기본.
- **git 4단계:** 작업폴더 → (add) 스테이징 → (commit) 로컬 → (push) 원격(GitHub).
- **gh repo create:** CLI로 원격 저장소를 만들고 바로 연결·push.
- **GitHub Pages:** 저장소의 정적 파일을 무료 웹사이트로. ***.github.io** 도메인. 주소는 소문자.
- **github.com vs github.io:** 코드 참고 vs 실제 화면. 둘은 다른 주소, 같은 GitHub.
- **공개 저장소 = 누구나 봄:** 올리기 전 민감 정보(IP·키) 제거. README는 명함처럼.

- **정적 사이트 (static site):** 서버 프로그램 없이 파일만으로 도는 사이트. 그래서 Pages가 공짜로 호스팅.
- **cron:** 정해진 주기로 명령을 자동 실행하는 스케줄러. * * * * * =매분.
- **幂등성 (idempotency):** 같은 작업을 여러 번 해도 결과가 한 번 한 것과 같음. 자동화·결제 둘 다의 핵심. (Layer 1·2에서 재등장)
- **HTML 주석 <!- - -> :** 화면엔 안 보이는 메모. 메타데이터 숨기기에 활용.

7. 비유 모음

- **단일 진실 공급원 = 칠판 하나:** 진행 상황을 칠판(roadmap.json) 한 곳에만 적는다. 사람들이 보는 게시물(화면)은 그 칠판을 베껴 그릴 뿐. 칠판만 고치면 모든 게시물이 갱신된다. 게시물마다 따로 고치면 어긋난다.
- **git push = 사진 인화 후 우편 발송:** add(앨범에 넣을 사진 고르기) → commit(앨범 한 장으로 확정) → push(그 앨범을 GitHub로 부치기).
- **github.com vs github.io = 주방 vs 식탁:** github.com은 재료·레시피가 쌓인 주방(코드), github.io는 손님이 받는 완성된 요리(화면).
- **cron = 알람 맞춰둔 집사:** 1분마다 logs/ 폴더를 들여다보는 집사. 새 게 있으면 알아서 처리하고, 없으면 가만히 있는다(幂등성).
- **幂등성 = 엘리베이터 버튼:** 이미 눌린 버튼을 또 눌러도 아무 일 없다. 한 번 누른 것과 같다.

8. 회고

자바는 안 찢지만, 오늘 한 게 더 작은 일이 아니다. 프로젝트를 처음으로 인터넷에 올렸고, GitHub Pages로 공개 사이트를 띄웠고, 자동 배포 파이프라인까지 만들었다. 이걸 전부 백엔드 실무에서 매일 하는 일이다 — git, 배포, 자동화.

배운 것 중 제일 값진 건 **幂등성**이다. 단어로만 알던 걸, 내 자동화 도구가 1분마다 멍청하게 커밋 쌓지 않게 만들면서 손으로 익혔다. 이게 몇 달 뒤 결제 원장의 "幂등성 키"로 돌아온다.

그리고 공개의 무게도 배웠다 — IP 하나 빼고, 표현 하나 다듬는 게, "남이 본다"는 전제에서 얼마나 중요한지.

내일부터는 logs/에 파일 두 개만 올리면 이 모든 게 자동으로 굴러간다. 추적 시스템이 깔렸으니, 이제 다시 코드로 — 증분 3, 커스텀 예외다.

한 줄: 오늘은 길(이정표)을 닦은 날이다. 길이 생겼으니, 이제 걷기만 하면 된다.